

The University of Waikato

Department of Computer Science

COMPX203 Computer Systems

Exercise 2 – C and WRAMP Programming

Due Date: 13 April 2022

Objectives

This exercise explores the relationship between high level programming languages, assembly code, and the system stack. You'll be writing programs using C and WRAMP assembly code, and interacting between the two using the system stack.

*It is highly recommended that you read this entire specification **before** beginning to write any code.*

Assessment

This exercise contributes **10%** towards your **internal grade**, to be assessed in two parts:

- The correctness of your source code, to be submitted via Moodle **(7%)**
- Your completion of an online quiz about the exercise, also on Moodle **(3%)**

Both of these parts must be completed on or before the due date.

This is an individual exercise; while you may discuss the assignment with others in general terms, you must write your own code, and complete the quiz by yourself.

The name of the file must match the following format:

ex[exercise number]_q[question number].srec

For example, the third question in this exercise would be called ***ex2_q3.srec***

Questions

1. Passing Parameters using the Stack

Create a new WRAMP assembly file called **ex2_q1.s**, and in that file create a subroutine called `print`. You don't need to declare a `main` label in this file, because we'll link against another file later on that contains the `main` subroutine of the program.

`print` is a function that takes a single integer parameter and prints it on the first serial port, then returns. The caller will read the switches and call `print` with the integer as argument. Your code should only print the value received by the caller on the first serial port. The maximum decimal value represented by the switches is 65535 (when all switches are on or 'up'). Since the Basys board has only four SSDs, we will send the value of the switches to the

first serial port instead. This will allow showing the full decimal number represented by the switches (four SSDs will show only up to 9999 decimal) on the serial port. **The value printed should be padded to the left with 0s if it is shorter than five characters. For example, if the number given is 1234, your code should print "01234".**

For testing, you should be able to change the switches positions and type *go* to see the decimal value of the switches on the screen (you need to enter *go* whenever you change the switches to see the new value on the screen, as the main object file reads the switches and calls `print` only once, then returns). *Hint: a five digit decimal number cannot be sent to the serial port as is, you have to send each digit separately.*

To form a complete program, your **ex2_q1.s** file needs to be assembled and linked with two other object files: one containing the `main` subroutine, and the other containing the collection of library functions listed at the end of this document. The library file (**lib_ex2.o**) and **main_q1.o** can be found at **/home/comp203/ex2/**.

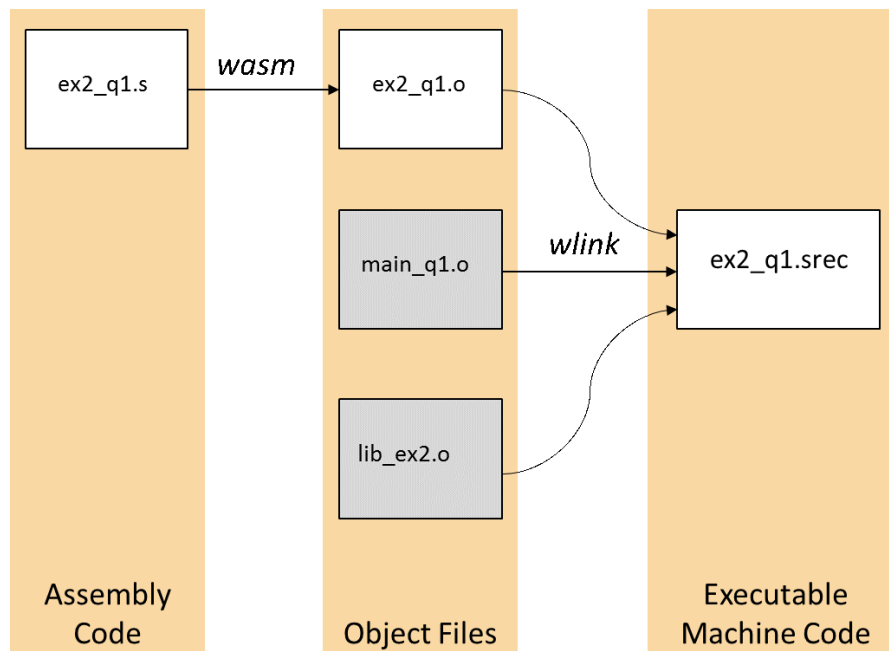


Figure 1: Layout of the program for question 1. Grey boxes are the files we provide.

Assemble and link your code using the commands:

```
wasm ex2_q1.s
```

```
wlink -o ex2_q1.srec  
ex2_q1.o  
/home/comp203/ex2/main_q1.o  
/home/comp203/ex2/lib_ex2.o
```

(wlink's arguments should be on a single line, separated by spaces)

Once you've created your S-Record, load and run your program. If it works correctly, you should see a decimal number appearing in either the "Serial Port 1" form if you're using *wsim*, or in the "remote" window if you're using a physical Basys board.

QUESTION 1 MUST BE WRITTEN IN WRAMP ASSEMBLY – DO NOT USE C

To ensure your program works correctly, be sure to follow the conventions described in the WRAMP manual (Chapter 2: Stack Guide) and lecture notes. Failing to do so will result in grade deduction(s), even if the program is working as required.

2. A Counter Implementation using C

C is a high-level language that offers many advantages over assembly code, enabling the use of familiar programming structures like for-loops, functions ("methods"), and if-statements.

Start by creating a C file called **ex2_q2.c**, which will contain a single function called `count`. This function takes two integer parameters, called `start` and `end`. It should look something like:

```
/**
 * Counts from "start" to "end" (inclusive),
 * showing progress on the seven segment displays.
 */
void count(int start, int end) {...}
```

When called, this function begins counting from the value `start`, and continues until it reaches the value `end` (both inclusive). As it counts, each number is shown on the seven segment displays as a four-digit base-10 number. If `start` is larger than `end`, it counts down; otherwise, it counts up.

To ensure that `start` and `end` are sensible values, check that they are both within the range $0 \leq n < 10000$. If either number falls outside this range, you should return without doing any counting. If `start` and `end` are equal, you can choose to either return without doing any counting, or set the SSDs to the value provided as though counting from `start` to the same number.

For example, if we call `count(6, 12)`, the function should output 06, 07, 08, 09, 10, 11, 12 to the seven segment displays, with a short delay between each number.

Hints:

- For the compiler to know *how* to call the library functions, you need to put the line
`#include "/home/comp203/ex2/lib_ex2.h"`
at the top of your C file, hard against the left margin. Don't copy/paste, because PDFs often contain non-ASCII characters that the compiler doesn't understand.
- See function descriptions at the end of the assignment.
- All variables need to be declared at the top of a block, e.g., the top of a function.

- Unlike in the previous lab exercise, the `writessd()` function in the **lib_ex2.o** library formats the value as a base-10 (decimal) number.
- Calling the `delay()` function after each call to `writessd()` will slow down the counting so that it is visible.
- You don't have to worry about the stack for this question, because the compiler will take care of it for you. Just call the necessary functions like you would in C# or Java.
- The WRAMP compiler is **wcc**. To *compile* your code, use the command:
wcc -S ex2_q2.c
 The "-S" tells the compiler to generate an assembly file as its output, which will be called `ex2_q2.s` in this case. Open up this file with a text editor, and see how it corresponds to your original C code.

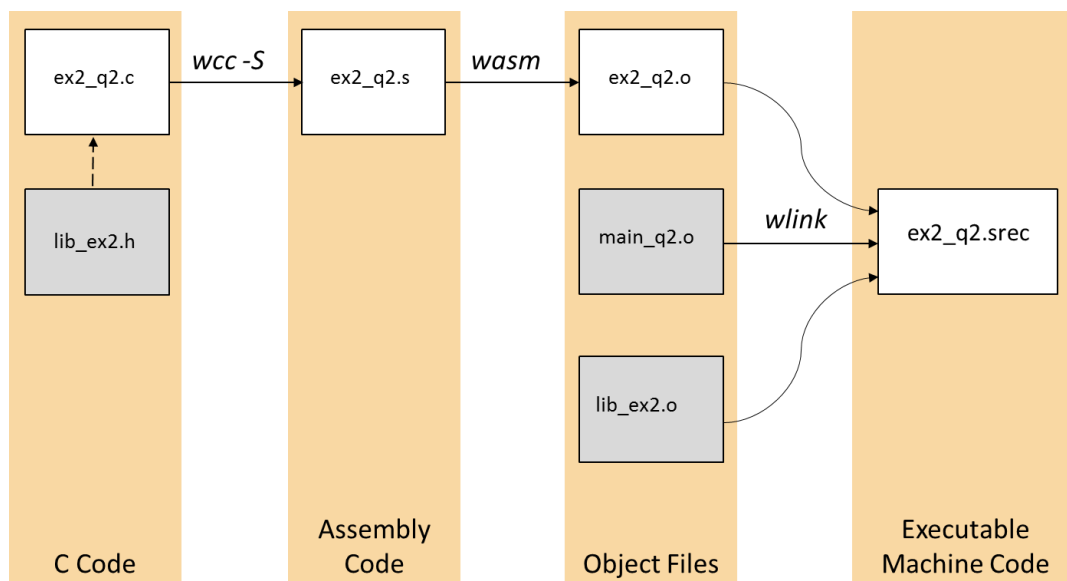


Figure 2: Layout of the program for question 2. Grey boxes are files we provide.

Next, complete the compile/assemble/link process according to the layout shown in Figure 2. The `main` subroutine is implemented in the file `/home/comp203/ex2/main_q2.o`. This `main` subroutine reads the value of the switches as two 8-bit numbers; the leftmost eight bits are passed to your C function as the parameter `start`, and the rightmost eight bits are passed as the parameter `end`.

When you type *go*, your program should count *from* the 8-bit number on the leftmost switches, *to* the number on the rightmost switches, in base 10. For example, if the leftmost switches were all up, and the rightmost switches were all down, your program should count *from 255 to 0*. You'll notice the leftmost seven-segment display will always be 0 - this is fine!

You can type *go* as many times as you like without resetting the board, which makes it easy to try other switch combinations.

3. Manually Calling `count()` from Assembly Code

Replicate the functionality of the program in Question 2, but with the values of the switches reversed: that is, the *rightmost eight bits* are now passed to your C function as `start`, while the *leftmost eight bits* are now passed as `end`. For example, if the leftmost switches were all down, and the rightmost switches were all up, your program should now count *from 255 to 0*.

To do this, create a new **WRAMP assembly** file called **ex2_q3.s**, and in that file create the `main` subroutine for the program. When assembled, this will serve as a drop-in replacement for the **main_q2.o** file provided and used in question 2. Do not modify any existing code; instead, just link against the unmodified **ex2_2.o** file as shown in Figure 3.

This means that the code in your **ex2_q3.s** file will need to call `readswitches` and `count` by following all the relevant subroutine conventions.

Hints:

- Make sure that you can run your program (i.e. by typing `go`) multiple times without resetting the board. This requires halting your program either by returning from `main`, or using the `syscall` instruction.
- The subroutine `main` is like any other: if you want to return from it after calling any other subroutines, you first have to save `$ra`.

QUESTION 3 MUST BE WRITTEN IN WRAMP ASSEMBLY, NOT C!

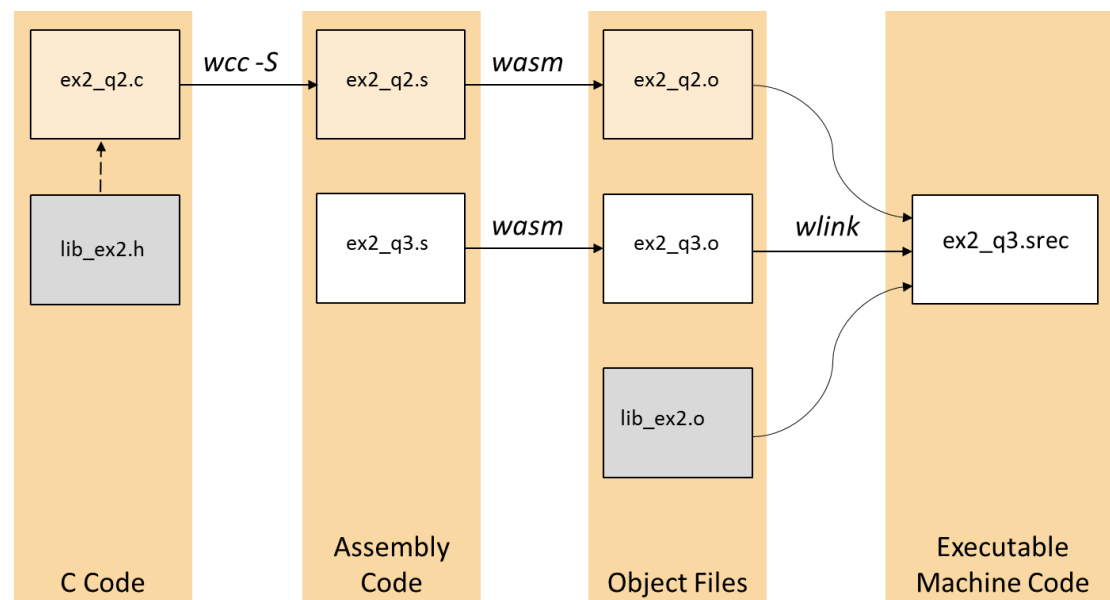


Figure 3: Layout of the program for Question 3. Grey boxes are files we provide, while light orange boxes are the files you created for Question 2.

Tips

This lab exercise is often seen as one of the most difficult in the course. Do yourself a favour and start early, make use of the supervised lab sessions, and ask for help sooner rather than later if you get stuck.

That being said, the programs are not huge. If you find yourself writing lots of code, stop and think about the problem again. Draw diagrams. Explore the debugging features of *wsim*, especially the “Memory (RAM)” form that shows the stack and `$sp`. Ask questions.

For reference, model solutions are (for each question):

1. 39 lines, only 14 of which are assembly instructions that don’t just back up registers onto the stack
2. 30 lines, including 14 that are either whitespace or a single curly bracket
3. 19 lines, only 13 of which are assembly instructions

If you’re writing significantly more than this, chances are that you’re doing something wrong.

Submission

You are required to submit all source files that **you** have written; you do not need to submit any files that we have provided, nor any files that have been generated by a tool, e.g., *wcc*, *wasm* or *wlink*. Each file must follow the naming convention indicated below.

For this exercise, the required files are:

- **ex2_q1.s** (Question 1)
- **ex2_q2.c** (Question 2)
- **ex2_q3.s** (Question 3)

These files must be compressed into a single “*tar gz*” archive called **firstName_lastName.tgz** (replace *firstName_lastName* with your own name) before being submitted to Moodle. You can create this archive from the terminal, using the command:

```
tar -cvzf firstName_lastName.tgz ex2_q1.s ex2_q2.c ex2_q3.s
```

Be especially careful about file extensions with this one – we’re not interested in your *ex2_q2.s* file, because we can easily generate that from your *ex2_q2.c* file (which we are very interested in).

Please be aware that a grade penalty applies if you do not submit your code in the correct format, including using the correct *file names* and *file extensions*. If you are unsure if what you’ve done is correct, please ask and we’ll help you out.

Library Functions (/home/comp203/ex2/lib_ex2.o)

void putc(int c)

Parameters: c The ASCII-encoded character to transmit

Returns: none

*Transmits a single ASCII character through the first serial port. This character will appear in **remote** if using the physical REX boards, or the **Serial Port 1** form if using the simulator. If the serial port is not ready to transmit, this function will block.*

int getc()

Parameters: none

Returns: The ASCII-encoded character that was read from the serial port

*Reads a single ASCII character from the first serial port. This is a character that was sent via **remote** if using the physical REX boards, or via the **Serial Port 1** form if using the simulator. This function will block until a character is received.*

void putstr(const char *s)

Parameters: *s A pointer to the string to be written to the serial port

Returns: none

*Transmits a null-terminated ASCII string through the first serial port. This string will appear in **remote** if using the physical REX boards, or in the **Serial Port 1** form if using the simulator.*

int readswitches()

Parameters: none

Returns: The value represented by the switches

Reads the current value represented by the switches into register. The lowest 16 bits (i.e. bits 0 to 15) will have the value of the corresponding switch (on or off), while the remaining bits (15 to 31) will be set to zero. The up (or 'on') position is represented as a binary '1'.

void writessd(int n)

Parameters: n The number to write to the seven-segment displays. $0 \leq n < 10000$.

Returns: none

Writes a number to the seven-segment displays, formatted as a four-digit decimal number.

void delay()

Parameters: none

Returns: none

Delays execution for approximately 135 ms.